

Automated Evaluation of LLM-Generated Quantum Computing Unit Tests

A framework for measuring how well AI models generate tests for quantum programs

QHack 8-Challenge · PennyLane · 5 Test-Generation Models · v3 Results

March 2026

What this work does in one sentence:

We built an automated pipeline that asks 5 AI models to write unit tests for 8 quantum computing challenges, then measures whether those tests are any good — using three simple scores: does it run, does it accept correct code, and does it catch bugs?

1. What we are trying to do

Writing unit tests for quantum programs is hard. Quantum algorithms produce probabilistic outputs, use unfamiliar APIs, and require precise numerical tolerances. The question we want to answer is:

"Can large language models automatically generate unit tests for quantum computing programs that are actually useful?"

To answer this we built a two-phase framework:

- **Phase 1** — Ask 5 different AI models to generate unit tests for 8 quantum programming challenges.
- **Phase 2** — Measure the quality of those tests with three scores: do they run, do they accept correct code, and do they catch bugs?

We evaluated **5 test-generation models** across **8 challenges**, producing **160 test files**. Each was run against **64 LLM-generated solutions** (labelled correct or wrong by an oracle) and **34 human ground-truth solutions**.

2. The data

Quantum challenges

8 challenges from the QHack quantum computing competition, implemented in PennyLane. Each has a template notebook and two official test cases with known correct outputs.

Human solutions

Five independent human solvers provided solutions (34 notebooks total). These are the ground truth — a good test must always pass on them.

LLM-generated solutions

Four AI models (Gemini, GPT-4.1, LLaMA-4, Qwen3) each generated two solutions per challenge — standard and retrieval-augmented — giving 64 solutions total. An oracle labels each one automatically:

Label	Count	%	Meaning
CORRECT	21	33%	Passes all official test cases
WRONG	23	36%	Fails at least one test case
UNKNOWN	20	31%	Could not be evaluated (syntax errors, deprecated APIs)

3. How the framework works

The diagram below shows the full pipeline.

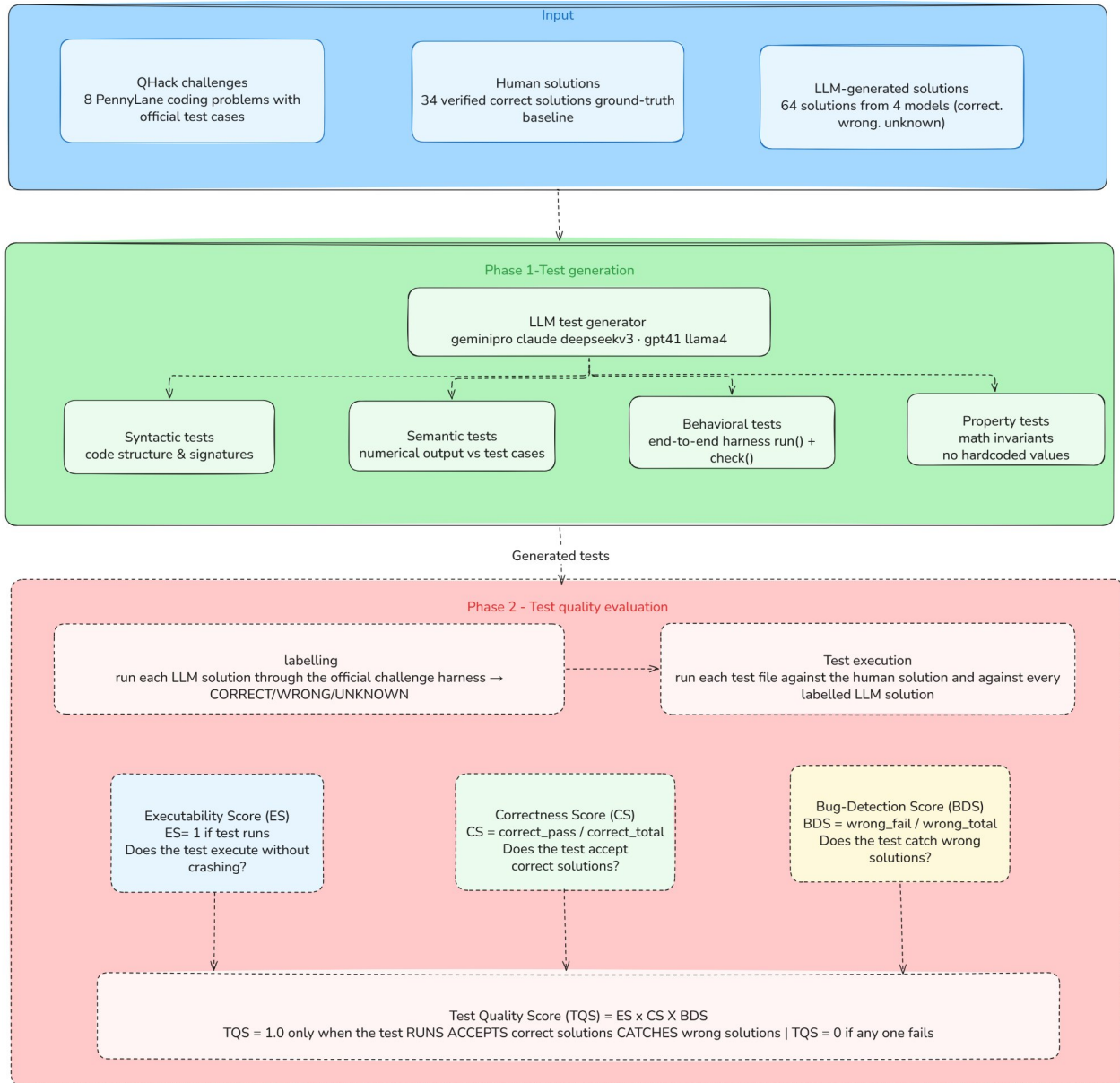


Figure 1. Framework architecture — inputs, Phase 1 (test generation), Phase 2 (evaluation), and quality scores.

Phase 1 — Test generation

For each challenge, the framework calls an AI model and asks it to generate four types of unit tests:

- **Syntactic** — check that the solution has the right function names and signatures.
- **Semantic** — call run() on official inputs and check the output matches official expected values.
- **Behavioral** — run the full harness: call run() then check() for every official test case.
- **Property** (new in v3) — check mathematical invariants such as probabilities summing to 1 or outputs being finite, without using any hardcoded expected values.

Key v3 fix: Previous versions told the AI to use the official test cases but the AI kept inventing its own expected values, causing every correct solution to fail the test. The fix was to inject the official test cases directly into the test file as a Python variable before the AI writes any code. The AI then reads from that variable instead of guessing.

Phase 2 — Evaluation

Each generated test file is run against the human solution and every labelled LLM solution inside a sandboxed environment. The results are used to compute three quality scores.

4. The three quality scores

Score	Full name	Formula	Question it answers	Ideal
ES	Executability Score	1 if test runs, else 0	Does the test run without crashing?	1.0
CS	Correctness Score	$\text{correct_pass} / \text{correct_total}$	Does it accept correct solutions?	1.0
BDS	Bug-Detection Score	$\text{wrong_fail} / \text{wrong_total}$	Does it catch wrong solutions?	1.0
TQS	Test Quality Score	$\text{ES} \times \text{CS} \times \text{BDS}$	Is the test good overall?	1.0

$$TQS = ES \times CS \times BDS$$

How to read TQS: It is 1.0 only when all three are satisfied at the same time. One zero kills the whole score. A test that crashes → TQS = 0. A test that rejects correct code → TQS = 0. A test that misses all bugs → TQS = 0.

5. Results

Overall scores averaged across all 8 challenges and 4 test types:

Model	ES	CS	BDS	TQS	Character
claude	1.00	0.56	0.44	0.00	Most balanced — highest CS
geminipro	0.91	0.38	0.53	0.00	Balanced — good BDS too
deepseekv3	1.00	0.22	0.78	0.00	Strong bug detection
gpt41	1.00	0.25	0.75	0.00	Strong bug detection
llama4	0.94	0.13	0.81	0.00	Too strict — low CS

Main finding — mutual exclusion:

TQS = 0.00 for every test file across all 5 models and 8 challenges. No test achieves both CS > 0 and BDS > 0 simultaneously. Tests are either too lenient (accept wrong code) or too strict (reject correct code) — never both calibrated at once.

Scores by test type

Breaking results down by test type reveals why TQS is always zero:

Test type	ES	CS	BDS	What this means
Syntactic	0.95	0.68	0.32	Accepts correct code well but rarely catches bugs

Semantic	1.00	0.17	0.82	Catches bugs but rejects correct code too often
Behavioral	0.97	0.05	0.95	Best at catching bugs, worst at accepting correct code
Property (new)	0.95	0.37	0.60	Most balanced type — invariant checks help both scores

Notable result: The new property-based tests (introduced in v3) are the most balanced test type — CS = 0.37, BDS = 0.60. This is because they check mathematical invariants that any correct solution satisfies, rather than exact numerical values. Claude's property tests reach CS = 0.75, the highest correctness score for any single model-type combination.

Improvement from v2 to v3

The v3 fix (injecting official test cases) produced measurable improvements in semantic test quality:

	v2 CS	v3 CS	Change	v2 BDS	v3 BDS	Change
Syntactic	0.65	0.68	+0.03	0.28	0.32	+0.04
Semantic	0.06	0.17	+0.11	0.88	0.82	-0.06
Behavioral	0.06	0.05	-0.01	0.91	0.95	+0.04
Property (new)	—	0.37	new	—	0.60	new

Semantic CS improved from 0.06 to 0.17 — a 3× increase — confirming that the injection fix works. The new property test type adds a genuinely more balanced option that neither v2 type provided.

6. Per-challenge results (claude)

The table below shows scores for the claude test-generation model on each challenge individually. Claude was chosen because it has the highest overall CS (0.56).

Challenge	ES	CS	BDS	Notable observation
chalet_random_gate	1.00	0.75	0.25	Best CS for any challenge
coffee_conundrum	1.00	0.50	0.50	Most balanced challenge
contextuality_dunes	1.00	0.25	0.75	Good bug detection
mach_zender_cabin	1.00	0.50	0.50	Balanced
QSP_swamp	1.00	0.50	0.50	Balanced
rainy_days_retreat	1.00	0.75	0.25	High CS — tests too lenient
travelling_eigentracks	1.00	0.25	0.75	0/8 correct LLM solutions — hardest challenge
triple_H_hotel	1.00	1.00	0.00	Perfect CS but zero bug detection

travelling_eigentracks is the hardest challenge: none of the 8 LLM-generated solutions was labelled correct by the oracle, meaning no AI model solved this Quantum Phase Estimation task correctly. Despite this, BDS remains high (0.75) because the tests do successfully reject all attempts.

triple_H_hotel with claude shows the clearest mutual-exclusion case: CS = 1.00 (tests accept every correct solution) but BDS = 0.00 (tests also accept every wrong solution). The tests are completely non-discriminative.

7. What this means and what to do next

The results show that current LLMs face a precision-recall tradeoff when generating quantum unit tests. The v3 fix improved semantic CS by 3× but did not resolve the fundamental tension between acceptance and rejection.

Three directions to improve TQS above zero:

- **Hybrid ensemble.** Use claude or geminipro for syntactic and property tests (high CS) and deepseekv3 or gpt41 for semantic tests (high BDS). A combined suite could achieve TQS > 0.
- **Property tests as the main type.** The new property test type is the most balanced (CS = 0.37, BDS = 0.60). Generating more property tests and fewer behavioral tests could improve overall TQS.
- **Multiple human solutions as reference.** Using all 5 human solutions as the CS baseline (rather than one) would make the correctness estimate more robust.

The framework is fully automated and reusable. Given any set of quantum challenges with official test cases and human solutions, it can evaluate the quality of tests generated by any LLM available through the OpenRouter API.